

CUSTOMER SHOPPING BEHAVIOR ANALYSIS

1. Project Overview

This project analyzes customer behavior using data from 3900 purchases across various product categories. The goal is to find useful insights into spending patterns, customer segments, product preferences and subscription behavior to guide strategic business decisions.

2. Dataset Summary

Rows: 3900

COLUMNS: 18

MISSING DATA: 37

3. Exploratory Data Analysis using Python

- Data Loading
- Initial Exploration
- MISSING DATA HANDLING
- COLUMN STANDARDIZATION
- FEATURE ENGINEERING
- DATA CHECK
- DATABASE INTEGRATION

```
[1]: import pandas as pd
df=pd.read_csv('customer.csv')

[2]: df.tail(2)
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	Brown	Summer	3.8	No	Express	No	No	24	Visa
3899	3900	52	Female	Handbag	Accessories	81	California	M	Beige	Spring	3.1	No	Store Pickup	No	No	33	Visa

```
[16]: from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
username = "postgres" # default user
password = "AGR1NGUPTA10469" # the password you set during installation
host = "localhost" # if running locally
port = "5432" # default PostgreSQL port
database = "customer_behavior" # the database you created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# Step 2: Load DataFrame into PostgreSQL
table_name = "customer" # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")

Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

[16]:

```
from sqlalchemy import create_engine

# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
username = "postgres" # default user
password = "AGRINDUPTA10469" # the password you set during installation
host = "localhost" # if running locally
port = "5432" # default PostgreSQL port
database = "customer_behavior" # the database you created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# Step 2: Load DataFrame into PostgreSQL
table_name = "customer" # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
Data successfully loaded into table 'customer' in database 'customer_behavior'.
```

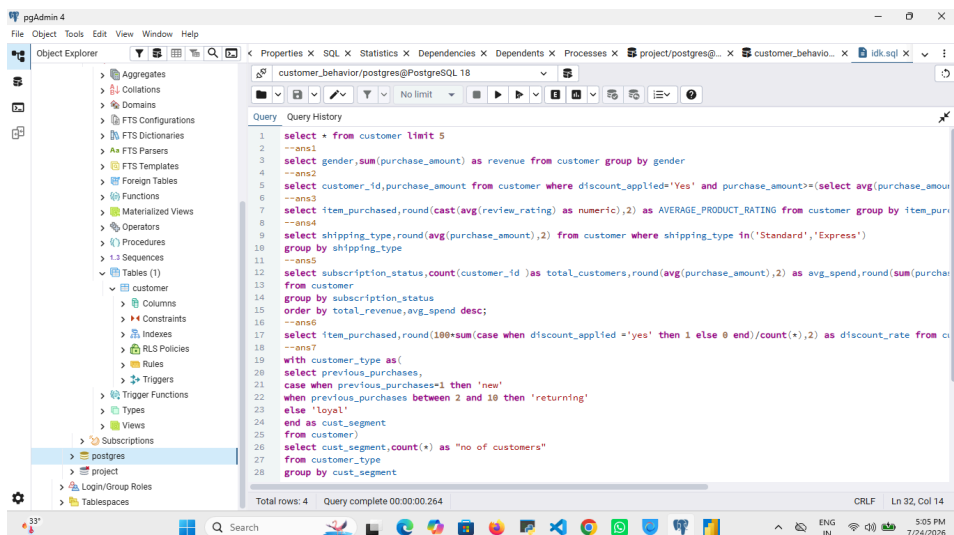
```
[4]: df.isnull().sum()
```

```
[4]: Customer ID      0
Age                  0
Gender               0
Item Purchased       0
Category             0
Purchase Amount (USD) 0
Location             0
Size                 0
Color                0
Season               0
Review Rating        37
Subscription Status   0
Shipping Type         0
Discount Applied      0
Promo Code Used       0
Previous Purchases    0
Payment Method        0
Frequency of Purchases 0
dtype: int64
```

4. SQL QUERIES

Structured data analytics to answer following 10 business questions

- Revenue by Gender
- High Spending Discount Users
- Top 5 products by rating
- Shipping Type Comparison
- Subscribers vs Non Subscribers
- Discount dependent products
- Customer Segmentation
- Top 3 products per category
- Repeat Buyers and Subscriptions
- Revenue by Age group



pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- > Aggregates
- > Collations
- > Domains
- > FTS Configurations
- > FTS Dictionaries
- > FTS Parsers
- > FTS Templates
- > Foreign Tables
- > Functions
- > Materialized Views
- > Operators
- > Procedures
- > Sequences
- > Tables (1)
 - customer
 - > Columns
 - > Constraints
 - > Indexes
 - > RLS Policies
 - > Rules
 - > Triggers
- > Trigger Functions
- > Types
- > Views
- > Subscriptions
- > postgres
 - > project
 - > Login/Group Roles
 - > Tablespaces

Properties X SQL X Statistics X Dependencies X Dependents X Processes X project/postgres@... X customer_behavio... X idk.sql X

customer_behavior/postgres@PostgreSQL 18

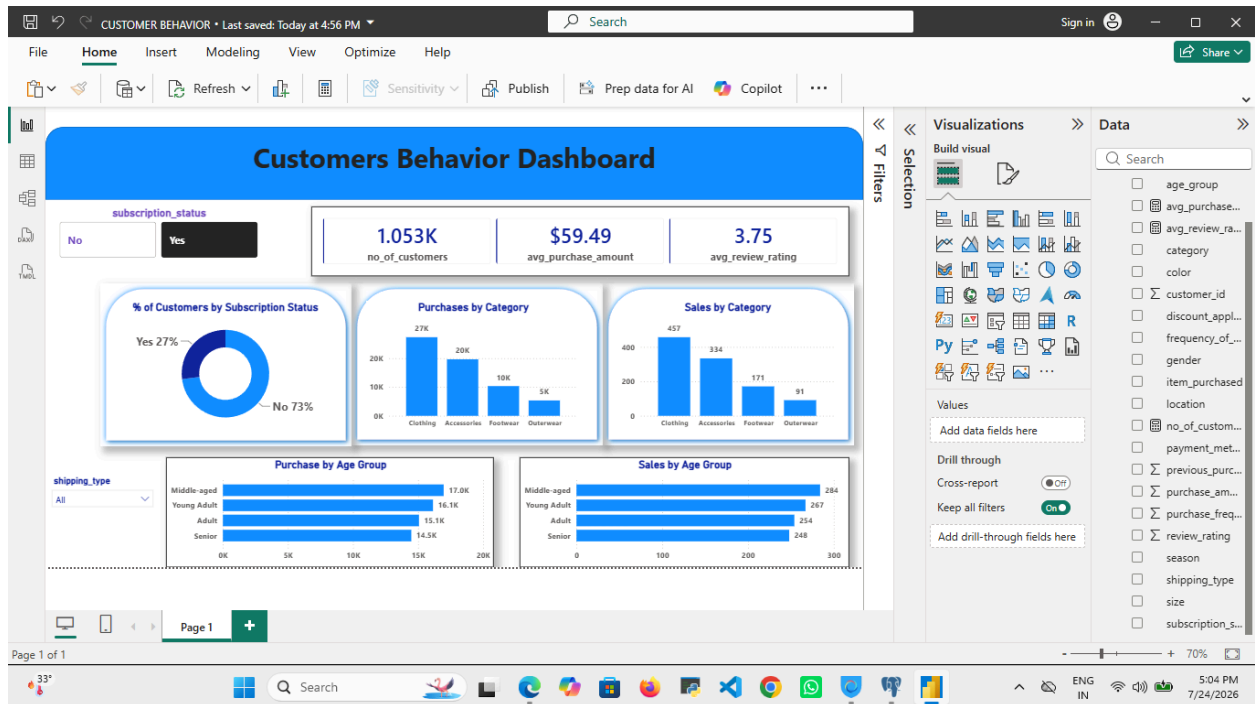
Query Query History

```
12 select subscription_status,count(customer_id) as total_customers,round(avg(purchase_amount),2) as avg_spend,round(sum(purchase
13 from customer
14 group by subscription_status
15 order by total_revenue,avg_spend desc;
16 --ans6
17 select item_purchased,round(100*sum(case when discount_applied ='yes' then 1 else 0 end)/count(*),2) as discount_rate from cu
18 --ans7
19 with customer_type as(
20 select previous_purchases,
21 case when previous_purchases=1 then 'new'
22 when previous_purchases between 2 and 10 then 'returning'
23 else 'loyal'
24 end as cust_segment
25 from customer)
26 select cust_segment,count(*) as "no of customers"
27 from customer_type
28 group by cust_segment
29 --ans8
30 with item_counts as(
31 select category,item_purchased,count(customer_id) as total_orders ,row_number() over(partition by category order by count(cu
32 from customer
33 group by category,item_purchased)
34 select item_rank,category,item_purchased,total_orders from item_counts where
35 item_rank<=3;
36 --ans9
37 select subscription_status,count(customer_id) as repeat_buyers from customer where previous_purchases >5 group by subscripti
38 --ans10
39 select age_group,sum(purchase_amount) as total_revenue from customer group by age_group order by total_revenue desc;
```

Total rows: 4 Query complete 00:00:00.264 CRLF Ln 32, Col 14

5:05 PM 7/24/2026

6. POWER BI DASHBOARD



7.BUSINESS RECOMMENDATIONS

- Boost Subscriptions – Promote exclusive benefits for subscribers.
- Customer Loyalty Programs – Reward repeat buyers to move them into the “Loyal” segment.
- Review Discount Policy – Balance sales boosts with margin control.
- Product Positioning – Highlight top-rated and best-selling products in campaigns.
- Targeted Marketing – Focus efforts on high-revenue age groups and express-shipping users.